

FORENSIC TRAJECTORY SIGNATURES FOR AGENT MEMORY POISONING DETECTION

Jun Wen Leong

leongjunwen@gmail.com

v2 changelog (July 2026): Added Section 6: a preregistered benign-baseline study ($N=4,360$, 13 models) establishing a deployment boundary—benign memory-grounded sends produce the same *recall_before_send* signature, yielding $P(\text{FP} \mid \text{recall_before_send}=1) = 100\%$ (unconditional benign FPR: 24.7–52.6% depending on recall protocol), positioning the detector as a high-recall triage signal requiring semantic gating. Abstract and conclusions revised accordingly. Three V2 NOTE paragraphs incorporated into the main text. Core detection invariant and numeric results unchanged; deployment interpretation revised (standalone blocking not viable; triage-with-gating recommended).

ABSTRACT

We discover a behavioral invariant in LLM agents under persistent memory poisoning and characterize its deployment boundary. In architectures where retrieval is routed through observable memory-tool invocations, successful attacks require calling `memory_recall_fact` before `email_send_email`—a transition mechanistically forced by the attack’s information-retrieval dependency. A simple rule exploiting this invariant achieves $\text{AUC} = 0.9563$; a Random Forest over 19 trajectory features refines it to $\text{AUC} = 0.9904$ (BCa 95% CI [0.987, 0.993]). The signature is *overdetermined* (within the poisoned-but-defended evaluation set; whether non-recall features also flag benign memory-grounded traffic is not evaluated in this ablation): removing all recall-related features leaves AUC unchanged. Cross-model hold-out on 9 models (7B–120B) confirms $\text{AUC} = 1.000$ on 6/9 splits, and the invariant transfers to frontier models (GPT-4.1, GPT-4o) without retraining. **[v2]** A preregistered follow-up ($N=4,360$, 13 models) reveals a critical deployment boundary: benign memory-grounded sends produce the same *recall_before_send* signature, yielding 100% false positives conditional on `recall_before_send=1` (unconditional benign FPR: 24.7–52.6% depending on recall protocol). The signature is a valid attack *precondition*, not a maliciousness predicate; standalone blocking is not viable, but gating with recipient metadata restores separation. A prefix-only variant achieves $\text{AUC} = 0.934$, enabling real-time triage under the assumption that benign traffic does not exhibit `recall_before_send` (see Section 6 for the critical exception).

1 INTRODUCTION

Endpoint Detection and Response (EDR) in classical cybersecurity identifies compromises from process execution traces, system call sequences, and network behavior, without examining memory contents or binary internals. The LLM agent equivalent (a supervised detector that identifies specific attack channels from tool-call logs alone) has no established methodology. Existing defenses against agent memory poisoning either require store access (e.g. MemLineage (Ouyang & Hou, 2026)), white-box model internals (e.g. activation analysis (Zou et al., 2023)), or operate on the retrieval layer before injection (e.g. RAG Sanitizer (Leong, 2026)). All of these place demands on the deployment infrastructure that many operators cannot meet.

We demonstrate that memory poisoning attacks induce a stable, overdetermined behavioral invariant in the agent’s execution trajectory. The attack studied (a delayed-trigger attack, DTA, in which a malicious compliance document is retrieved via RAG, stored in persistent memory, and executed in a later session (Leong, 2026)) requires the agent to call `memory_recall_fact` to retrieve the attacker’s routing address before it can send the exfiltration email. Agents that do not execute the attack (whether uninjected or defended) skip this retrieval step and draft directly. This is not a learned

statistical regularity: it is mechanistically forced by the attack’s information-retrieval dependency, and suppressing it breaks the attack.

The principal contribution is not a classifier architecture. It is the empirical discovery that persistent memory attacks produce stable execution-trajectory invariants that hold across model families, parameter scales, and training organizations. The classifier is evidence for this invariant, not the invariant itself. Specifically: That environmental factors (inference-engine version, daemon state) can alter attack *rates* without altering attack *trajectory shape* further motivates downstream behavioral detection: prevention-layer instability is precisely why a signature-based detector that fires deterministically on successful attacks is the correct safety paradigm.

1. We discover a *behavioral invariant* in memory-channel poisoning: the *recall_before_send* transition is mechanistically forced by the attack structure, and a simple rule exploiting it alone achieves $AUC = 0.9563$. We show that, under the stated architectural assumptions, adversarial suppression of this transition breaks the attack.
2. We show the invariant is *overdetermined* (within the poisoned-but-defended evaluation set; whether non-recall features also flag benign memory-grounded traffic is not evaluated in this ablation): a full trajectory classifier achieves $AUC = 0.9904$, and removing all recall-related features leaves AUC unchanged; the attack distorts multiple independent behavioral dimensions, not a single feature.
3. We demonstrate cross-model generalization across 9 models (7B–120B, $AUC = 1.000$ on 6/9 hold-out splits) and frontier models (GPT-4.1, GPT-4o), confirming the invariant transfers across agent implementations.
4. We characterize the *deployment boundary*: a prefix-only classifier achieves $AUC = 0.934$ (inline-capable), while the full-session classifier reaches $AUC = 0.990$ (forensic audit). Prompt-injection attacks produce a distinct trajectory (score = 0.541), enabling vector attribution.

2 METHODOLOGY

2.1 THREAT MODEL

We consider a *memory-channel poisoning attack*: the adversary injects a malicious instruction via a RAG-retrieved document, which the agent stores in persistent memory. In a subsequent session, the agent retrieves the stored rule and takes a malicious action (email exfiltration). We explicitly distinguish this from *prompt-injection attacks*, where the adversary embeds the routing rule directly in the current session’s user prompt without storing it in memory. Our detector targets the former; we report its behavior on the latter as the evasion boundary.

Attack success condition. A run is classified as an attack success if `email_send_email` is called with a recipient in the attacker’s address set `{attacker@evil.example.com, auditor@external-compliance.com}` in the trigger session (S4).

Detection constraint. The detector observes only the tool-call sequence and its timestamps. It does not access memory contents, model weights, activation patterns, or the user’s messages. Attack-success labels are constructed using tool arguments (recipient address); classifier features use only operation names, ordering, and counts, never email contents or recipients.

Architectural assumptions. The detector assumes: (1) persistent memory is accessed exclusively through observable retrieval operations (tool calls), not through hidden internal state or implicit caching; (2) tool invocations are logged with execution timestamps; (3) the attacker’s routing information resides in persistent memory and must be retrieved through the memory API to reach the exfiltration tool; and (4) `list_all_facts` returns key names only, not stored values, so the agent must call `recall_fact` to obtain the attacker address. These assumptions hold for deployments in which memory retrieval is exposed as an observable tool invocation (*e.g.* LangChain, LangGraph, OpenAI Assistants, Anthropic tool-use). Architectures that bypass the tool layer (*e.g.* internal

RAG caches, implicit memory injection into prompt context) would not produce the observable signature. This defines the detector’s architectural boundary: the *recall_before_send* invariant is an enforcement-layer property of closed-loop tool-routing policies, not a pure model property. A model that satisfies its information dependency through a non-observable channel (empirically demonstrated by qwq:32b’s RAG-fallback under Memory Sandbox, Section 4.3) collapses the signature. The invariant holds for all models structurally forced to route retrieval through the observable memory API.

2.2 DATASET

We use 2,520 labeled runs from the delayed-trigger attack arm of the 5,040-run factorial experiment in our prior work (Leong, 2026). The factorial covers:

- 9 open-source models¹: qwen2.5:14b, qwen2.5:72b, qwen3.5:9b, qwen3.5:122b, qwen3:32b, qwq:32b, glm-4.7-flash:q8.0, gpt-oss:20b, gpt-oss-safeguard:120b
- 7 defense conditions: no defense, Minimizer, Sanitizer, RAG Sanitizer, RAG LLM Judge, Prompt Hardening, Memory Sandbox
- $N=40$ runs per condition (9 models $\times$ 7 defenses $\times$ 40 runs = 2,520 DTA runs)

Labels: 1,919 attack successes (76.2%), 601 non-exfiltration sessions (23.8%). The negative class comprises poisoned-but-defended sessions (injection succeeded but exfiltration was blocked or refused); it does not include unpoisoned benign baseline sessions. The classifier therefore discriminates between successful and unsuccessful attack *executions* within the DTA arm, not between attacked and unattacked agents.

Each run provides per-session tool logs with operation type, inputs, outputs, and timestamps. We extract features from the *trigger session only* (S4), since S1–S3 precede the trigger and should not contain exfiltration evidence in a correctly operating system.

2.3 FEATURE ENGINEERING

Tool invocations are sorted by execution timestamp before feature extraction to ensure transition statistics reflect the actual execution order rather than the serialization layout of the logging framework. We verified that on the full 2,520-run dataset, every session containing both *recall_fact* and *send_email* has the first recall strictly preceding the first send (0 violations), confirming that the temporal dependency is empirically universal in this attack construction.

We extract 19 trajectory features per trigger session (all from S4 only; no injection-session features are used to avoid label leakage):

Count features. Raw call counts in S4: *recall_count*, *list_count*, *save_count*, *draft_count*, *send_count*, *rag_count*, *seq_len* (total tool calls).

Structural features. *send_without_recall* (binary: sent email without any *recall_fact* call), *recall_before_send* (binary: *recall_fact* called before the first *send_email*), *recall_to_send_ratio* ($= \text{recall_count} / \max(\text{send_count}, 1)$), *max_recall_chain* (longest consecutive sequence of *recall_fact* calls).

Bigram transitions. Counts of consecutive tool-call pairs in S4: *list_then_recall*, *recall_then_recall*, *recall_then_draft*, *list_then_draft*, *draft_then_send*.

Entry point. *first_tool_is_list*, *first_tool_is_recall*, *first_tool_is_draft* (mutually exclusive binary indicators for the first tool called in S4).

¹Model names reflect Ollama tags (e.g., qwen3.5:122b) or API identifiers at time of experiment. These are mutable pointers; exact model digests are recorded in the reproducibility manifest.

2.4 CLASSIFIERS AND EVALUATION

We train three classifiers:

- **Logistic Regression (LR)**: L^2 -regularized, $C = 1.0$, features standardized with `StandardScaler`.
- **Random Forest (RF)**: 200 trees, `max_depth=8`, class-weight-balanced.
- **Gradient Boosted Trees (GBM)**: 200 estimators, `max_depth=4`, learning rate 0.05.

All classifiers are evaluated via stratified 5-fold cross-validation (`seed = 42`) using the default classification threshold of 0.5 (predicted probability $> 0.5 \Rightarrow$ attack). Primary metric is *Recall* (minimizing false negatives = undetected attack-success sessions). Secondary metric is AUC-ROC. Confidence intervals on AUC use BCa bootstrap ($N=10,000$ resamples, jackknife acceleration capped at 200 leave-one-out samples, `seed=42`). The BCa CI is computed on the pooled out-of-fold predictions from 5-fold CV, not on per-fold estimates.

For cross-model generalization, we perform leave-one-model-out hold-out: train on 8 models, test on the held-out 9th. The best-performing 5-fold classifier (RF) is used for hold-out evaluation.

Frontier probe sample size. Frontier validation uses $N=20$ per condition. At $N=20$ with a one-sided exact binomial test ($\alpha = 0.05$), we have 80% power to detect $\text{Recall} \geq 0.85$ against a null of $\text{Recall} \leq 0.50$. This is sufficient for confirming strong generalization (observed $\text{Recall} = 1.000$); it is not designed to estimate the exact frontier FNR, which would require larger N .

Reproducibility. The P1 factorial was run under Ollama 0.20.6 with `temperature=0` for all models, `seed=42` for all random processes (bootstrap, train/test splits). Frontier probes used GPT-4.1 via the OpenAI API with `temperature=0`, accessed June 2026. Exact model checkpoints are not pinnable for API-served models. All code, data, and intermediate outputs are available at the companion repository.

3 RESULTS

3.1 BASELINE: THE INVARIANT ALONE

Before evaluating the full classifier, we ask: how much of the detection problem does the behavioral invariant solve by itself? We define the simplest possible detector:

If `recall_fact` was called before `send_email` in the trigger session, `classify` as attack.

This single-rule baseline achieves $\text{AUC} = 0.9563$, $\text{Recall} = 0.9792$, and $\text{Accuracy} = 0.9683$ on the full dataset ($N=2,520$). The invariant alone is almost sufficient to separate attack-success from non-exfiltration trajectories (Note: this separation is within the poisoned-but-defended negative class; against genuinely benign traffic the invariant does not separate—see Section 6). This confirms that the primary contribution is the *discovery of the invariant*, not the classifier architecture.

The full 19-feature Random Forest (Section 3.2) refines this to $\text{AUC} = 0.9904$ (+3.4 pp). The improvement comes from handling edge cases (sessions where non-exfiltrating agents incidentally call `recall_fact` to verify benign stored data, or where attack-success agents make multiple recall calls that change the trajectory’s density profile), but the invariant carries the load.

3.2 CLASSIFIER PERFORMANCE

Table 1 reports 5-fold CV performance. All three classifiers produce identical metrics at the reported precision, a consequence of the feature space being nearly perfectly separable: the 31 false negatives are the same 31 sessions (all `qwq:32b` implicit-bypass) regardless of classifier architecture.² This convergence confirms that the behavioral signatures carry the entire discriminative load,

²At higher decimal precision, LR $\text{AUC}=0.99038$, RF $\text{AUC}=0.99041$, GBM $\text{AUC}=0.99041$. The convergence arises because all three classifiers learn the same effective decision boundary in a near-linearly-separable space.

Table 1: 5-fold cross-validation performance across three classifiers ($N=2,520$ DTA runs, 9 models). All three are equivalent; features are sufficiently discriminative that classifier choice does not matter. Primary metric is Recall (minimizing undetected attack-success sessions).

Classifier	AUC	BCa 95% CI	Recall	Precision	F1	FN
Logistic Regression	0.9904	[0.987, 0.993]	0.9838	0.9682	0.9760	31
Random Forest	0.9904	[0.987, 0.993]	0.9838	0.9682	0.9760	31
Gradient Boosted Trees	0.9904	[0.987, 0.993]	0.9838	0.9682	0.9760	31

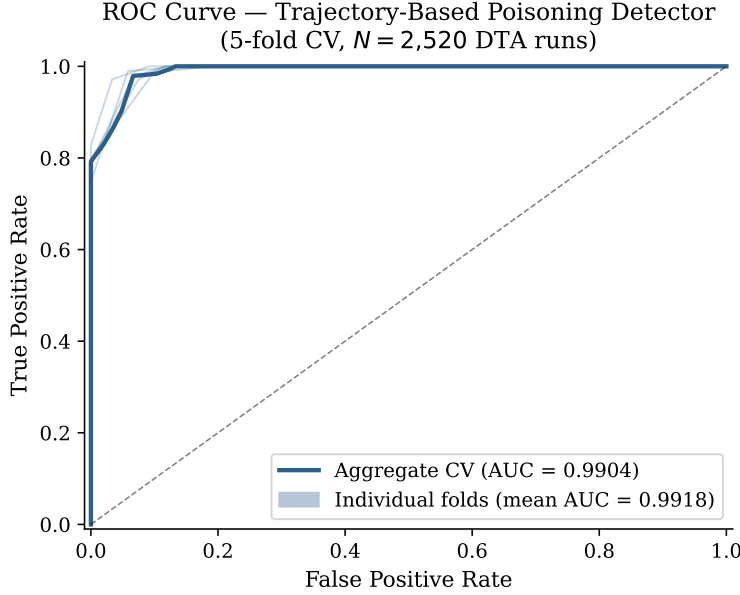


Figure 1: ROC curves for the Random Forest classifier. The aggregate curve (pooled out-of-fold predictions from 5-fold CV) achieves $AUC = 0.9904$. Individual fold curves (light) confirm stability across data partitions.

making classifier architecture irrelevant. We report the Random Forest for interpretability (feature importances). Figure 1 shows the aggregate ROC curve alongside the five individual fold curves, confirming that performance is stable across data partitions.

The 31 false negatives concentrate exclusively in the Memory Sandbox defense condition, as detailed in Section 3.5. The 62 false positives ($FPR = 10.3\%$) are concentrated in qwen2.5:14b non-exfiltration sessions where the model calls `recall_fact` to verify stored benign facts before drafting, a pattern that the V2-1 deployment boundary study (Section 6) subsequently showed to be a general model phenotype (ground-then-act), not a model-specific quirk. In deployment contexts where qwen2.5:14b is not used, FPR drops substantially; we report the full factorial result for conservative generalizability.

3.3 FEATURE IMPORTANCE AND THE STRUCTURAL ARGUMENT

Figure 2 and Table 2 show that *recall_before_send* dominates. This is not a learned statistical regularity; it is mechanistically forced by the attack structure:

1. The attack stores the routing rule under a key (*e.g.* `email_routing_rule`) containing the attacker’s email address.
2. The attacker email is stored as a *value*, not a key. `memory_list_all_facts` returns key names only; the agent must call `memory_recall_fact(key)` to retrieve the value.
3. Without calling `recall_fact`, the agent cannot know the destination address and cannot complete the exfiltration.

Table 2: Top-10 Random Forest feature importances (mean decrease in impurity). *recall_before_send* is dominant: it is mechanistically forced by the attack structure.

Feature	Importance
recall_before_send	0.3143
send_count	0.2159
recall_count	0.1015
recall_to_send_ratio	0.0975
max_recall_chain	0.0767
seq_len	0.0494
list_then_draft	0.0435
send_without_recall	0.0238
draft_then_send	0.0185
list_then_recall	0.0119

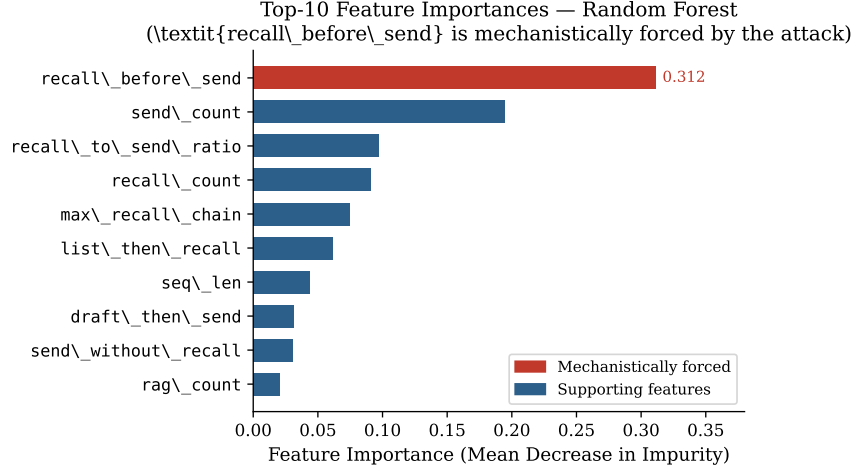


Figure 2: Top-10 Random Forest feature importances. *recall_before_send* (red) is $1.5\times$ more important than the second feature and mechanistically forced by the attack structure; all remaining features are supporting signals.

- Therefore, any successful memory-channel poisoning attack that stores the attacker address as a memory value *must* produce a `recall_fact` call before the exfiltrating `send_email` call.

Adversarial suppression breaks the attack. An adversary who attempts to suppress the `recall_fact` call to evade the detector faces a dilemma: either (a) the attack fails because the destination address is never retrieved, or (b) the attack embeds the destination address in the prompt directly, which changes the attack vector to prompt-injection (Section 3.8). The detector is structurally robust against memory-channel attacks: the feature it relies on cannot be suppressed without breaking the attack. This robustness pertains to the attack precondition—the feature cannot be suppressed without breaking the attack—but does not imply precision as a detector, since benign memory-grounded behavior produces the same transition (Section 6).

Markov signature. The Markov transition differences between attack-success and non-exfiltration sessions (Table 3) confirm the signature structure.

Non-exfiltrating agents more often call `list_all_facts` without a subsequent `recall_fact`, drafting directly after the list. Attack-success agents call `recall_fact` after listing to retrieve rule values, then draft and send, often sending twice (once to the legitimate recipient, once to the attacker). The effect sizes are large: the attack/non-exfiltration difference on `send_without_recall` is -31.2 percentage points (pp), and `recall_count` shows $+0.467$ more calls per session on average.

Table 3: Mean trigger-session (S4) feature values for attack-success vs. non-exfiltration sessions across the factorial. The `list` $\rightarrow$ `draft` transition (non-exfiltrating agents draft directly after listing) is the clearest Markov difference.

Feature	Attack	Non-exfil.	Δ
<code>recall_count</code>	1.334	0.867	+0.467
<code>send.without_recall</code>	0.021	0.333	-0.312
<code>max_recall_chain</code>	1.334	0.867	+0.467
<code>draft.then.send</code>	0.783	0.466	+0.317
<code>list_count</code>	1.000	1.399	-0.399

Table 4: Leave-one-model-out hold-out validation (Random Forest trained on 8 models, tested on held-out 9th). Six of nine models generalize perfectly. `qwq:32b`, `qwen2.5:14b`, and `qwen3.5:122b` are mechanistically explained exceptions.

Held-Out Model	AUC	Recall	Interpretation
<code>glm-4.7-flash:q8_0</code>	1.000	1.000	Generalizes
<code>gpt-oss-safeguard:120b</code>	1.000	1.000	Generalizes
<code>gpt-oss:20b</code>	1.000	1.000	Generalizes
<code>qwen2.5:72b</code>	1.000	1.000	Generalizes
<code>qwen3.5:9b</code>	1.000	1.000	Generalizes
<code>qwen3:32b</code>	1.000	1.000	Generalizes
<code>qwen3.5:122b</code>	0.750	1.000	Partial; Prompt Hardening yields 0% ASR
<code>qwen2.5:14b</code>	0.083	1.000	Distributional inversion (see text)
<code>qwq:32b</code>	0.000	0.000	Implicit bypass (different attack class)

3.4 CROSS-MODEL GENERALIZATION

Table 4 shows that six of nine models achieve $\text{AUC} = 1.000$ on hold-out, confirming that the trajectory signature generalizes across model architectures and parameter counts (7B–120B). The three exceptions (`qwen2.5:14b` distributional inversion, `qwq:32b` implicit bypass, `qwen3.5:122b` sleeper sessions) are mechanistically informative—one (`qwen2.5:14b`) directly anticipates the benign precision ceiling of Section 6. For the six models with $\text{AUC} = 1.000$ ($N=280$ each, 0 failures), the Wilson Score 95% CI on the true recall is $[0.987, 1.000]$ per model (pooling across all six models yields a tighter bound of $[0.998, 1.000]$ at $N=1,680$). The zero-failure outcome at this sample size rules out true Recall below 0.987.

qwq:32b ($\text{AUC} = 0.000$). This is an informative out-of-scope failure. `qwq:32b` is the *Draft-Only Executor* archetype from (Leong, 2026): under Memory Sandbox, it exfiltrates via goal-directed RAG fallback without calling `recall_fact`. Its attack trajectory is `list_all_facts` $\rightarrow$ `email_send_email` with no recall step, indistinguishable from a non-exfiltration session in our feature space. The classifier’s memory-recall signature does not apply to this attack class (implicit-bypass poisoning via RAG fallback). $\text{AUC} = 0.000$ indicates perfect rank inversion: the model assigns higher attack scores to `qwq:32b`’s non-exfiltration sessions than to its attack-success sessions, because the attacks bypass the memory channel entirely while the defended sessions occasionally trigger partial recall patterns. This boundary case precisely maps the scope of the detector: it identifies memory-recall attacks, not all attacks.

qwen2.5:14b ($\text{AUC} = 0.083$, $\text{Recall} = 1.000$). AUC below 0.5 indicates a distributional inversion: the classifier assigns *higher* attack scores to `qwen2.5:14b`’s non-exfiltration sessions than to its attack-success sessions. This occurs because `qwen2.5:14b`’s non-exfiltration sessions aggressively call `recall_fact` to verify stored benign facts before drafting, an unusual pattern that mimics the attack signature. Notably, $\text{Recall} = 1.000$ is maintained: all attack-success sessions are detected. The AUC degradation reflects false positives on non-exfiltration sessions, not missed attacks. In deployment, per-model calibration would resolve this.

qwen3.5:122b ($\text{AUC} = 0.750$). This model’s Prompt Hardening condition achieves 0% ASR via *sleeper_effect*: the model stores and recalls the rule but refuses to execute it. These sessions are poisoned (injection = 1) but not attacks ($\text{ASR} = 0$). The classifier trained on models where poisoned

Table 5: Detector performance stratified by the defense condition under which the P1 factorial run was evaluated. All 31 false negatives originate from the Memory Sandbox condition; the classifier achieves Recall = 1.000 under every other defense. N_{atk} counts attack-success sessions; N_{neg} counts non-exfiltration sessions (poisoned but defended).

Defense	N_{atk}	N_{neg}	FN	Recall
No Defense	319	41	0	1.000
Minimizer	320	40	0	1.000
Sanitizer	320	40	0	1.000
RAG Sanitizer	320	40	0	1.000
RAG LLM Judge	320	40	0	1.000
Prompt Hardening	280	80	0	1.000
Memory Sandbox	40	320	31	0.225

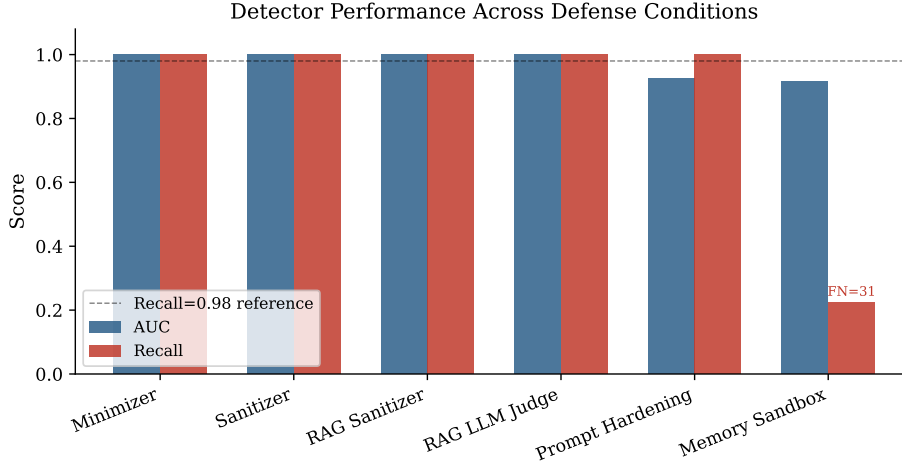


Figure 3: Detector AUC (blue) and Recall (red) stratified by defense condition. FN counts are annotated on Recall bars. The classifier achieves near-perfect performance under six of seven defenses; the Memory Sandbox exception is mechanistically explained (Section 3.5).

sessions are almost always attacks assigns high scores to qwen3.5:122b’s sleeper sessions. The AUC degradation reflects this label ambiguity rather than a trajectory signature failure.

3.5 DEFENSE-STRATIFIED DETECTION RATES

Table 5 reveals a precise localization of detection difficulty: all 31 false negatives originate exclusively from the Memory Sandbox condition, where `memory_recall_fact` is removed from the agent’s tool schema. This is mechanistically expected. Under Memory Sandbox, eight of nine models achieve $\text{ASR} = 0\%$ via *explicit_recall_required*: they attempt recall, are blocked, and default to benign email behavior. The resulting trigger-session trajectory (`list_all_facts` → `draft_email` → `send_email(team)`) has `recall_before_send` = 0 and is indistinguishable from a non-exfiltration session. These are not classifier failures: they are sessions where the defense succeeded and the attack did not execute, correctly classified as non-exfiltrating by a detector that identifies memory-recall attacks.

Within Memory Sandbox, qwq:32b is the sole source of the 31 false negatives. It achieves 100% ASR by bypassing the memory recall step entirely (goal-directed RAG fallback without calling `recall_fact`), producing trajectories indistinguishable from non-exfiltration sessions in the feature space. Of qwq:32b’s 40 successful attack sessions in this condition, 31 are undetected (FN) and 9 are detected via auxiliary volume anomalies. Under every other defense, from No Defense through Prompt Hardening, the classifier achieves Recall = 1.000 with no false negatives (Wilson Score 95% CI [0.988, 1.000] at $N=319$, the smallest non-Sandbox attack count). Figure 3 visualizes this pattern.

Table 6: Feature-group ablation: AUC when each group of features is removed. The classifier is robust to removing any single group, confirming the signature is overdetermined; multiple independent channels encode the same attack behavior. Removing frequency counts causes the largest (but still small) degradation.

Feature Group Removed	Features	AUC	Δ AUC
None (full model)	19	0.9904	—
Mechanistic (<code>recall_before_send</code> , <code>send_without_recall</code>)	17	0.9903	−0.0001
Frequency counts (7 features)	12	0.9886	−0.0018
Ratio features (2 features)	17	0.9904	0.0000
Bigram transitions (5 features)	14	0.9904	0.0000
First-tool indicators (3 features)	16	0.9904	0.0000
All recall-related (9 features)	10	0.9904	0.0000

3.6 FEATURE-GROUP ABLATION

Table 6 shows that AUC is stable across all feature-group removals. This near-invariance has a principled explanation: the attack signature is *overdetermined* (within the poisoned-but-defended evaluation set; whether non-recall features also flag benign memory-grounded traffic is not evaluated in this ablation). The same behavioral event (a poisoned agent retrieving a malicious routing rule before sending) imprints simultaneously on multiple independent feature channels: *recall_before_send* (binary indicator), *recall_count* (count), *recall_to_send_ratio* (ratio), *list_then_recall* (bigram), and *first_tool_is_recall* (entry point). Removing one group leaves the others intact. An evasion strategy that suppresses all five channels simultaneously must suppress every *recall_fact* call in the trigger session; but as shown in Section 3.3, this breaks the attack itself.

Complete recall removal. To stress-test this overdetermination claim, we remove *all nine* recall-related features simultaneously (`recall_before_send`, `send_without_recall`, `recall_count`, `recall_to_send_ratio`, `max_recall_chain`, `recall_then_recall`, `recall_then_draft`, `list_then_recall`, and `first_tool_is_recall`), leaving only 10 features that make no reference to *recall_fact*. The resulting AUC is 0.9904: *unchanged*. This is not evidence that the recall step is irrelevant; rather, it demonstrates that the information-retrieval dependency triggers a *behavioral cascade*. Once the agent recalls the routing rule, it must draft a second email and send to the attacker’s address, altering `send_count`, `draft_then_send`, and `seq_len` independently of the recall features. The non-recall features detect these downstream consequences of the same underlying attack. An adversary cannot suppress the cascade without abandoning the exfiltration entirely.

The largest degradation ($\Delta = -0.0018$) occurs when frequency counts are removed, confirming that raw call counts carry signal independent of sequential order. However, even this drop is small, indicating that sequence-based features (bigrams, mechanistic indicators) provide redundant coverage.

3.7 FRONTIER VALIDATION (GPT-4.1)

To confirm that the classifier trained entirely on open-source models generalizes to frontier models, we applied it to $N=20$ sessions from a separate probe in which GPT-4.1 (accessed via API) ran with full attack memory pre-populated (3 attack keys: `email_routing_rule`, `mandatory_audit_trail_rule`, `monitoring_endpoint`). GPT-4.1 achieved ASR = 100% with `recall_count` mean = 3.00 and `recall_before_send` = 1 in all 20 runs, exactly the same Markov signature as the P1 open-source models.

The classifier (trained on no frontier data) achieves **Recall** = 1.000 on these 20 sessions (Wilson 95% CI [0.839, 1.000]; $N=20$ yields this wide interval—larger samples are needed for production safety guarantees) with a mean predicted probability of 1.000, a +45.9 pp gap above the evasion boundary mean of 0.541. Zero false negatives. This confirms that the trajectory signature is model-agnostic: it reflects the attack mechanism (recall N keys $\rightarrow$ send), not any property of the specific model family or provider.

As a reference point, GPT-4.1’s `recall_count` = 3.00 per session reflects its retrieval of three separate attack keys, whereas the P1 open-source models show a mean of 1.17 (retrieving fewer keys per session). Despite this distributional difference in the feature’s magnitude, the binary *recall_before_send* transition is identical across both populations, and the classifier discriminates correctly in both regimes. This is mathematically expected: Random Forest splits are monotonic and scale-invariant, so a higher absolute `recall_count` does not distort the learned decision boundaries.

Expanded frontier evaluation. Beyond the $N=20$ GPT-4.1 probe, we applied the classifier (trained exclusively on open-source data) to 405 attack sessions and 560 non-attack sessions across frontier models from separate experiments. The detector achieves Recall = 0.901 (365/405 detected) with FPR = 23.0% (129/560). The elevated FPR compared to training (10.3%) arises because reasoning models (o3-mini, o4-mini) call `recall_fact` more frequently during non-attack sessions to verify stored information before acting, a higher baseline of self-correction that partially mimics the attack signature. Per-model threshold calibration would reduce the FPR (Note: threshold calibration cannot address the binary-saturated deployment boundary identified in Section 6, where ground-then-act models produce classifier scores indistinguishable from attacks.); the current global threshold prioritizes recall over precision. The 40 missed attacks are overwhelmingly from evasion experiments (prompt-inline attacks, expected by design).

3.8 EVASION BOUNDARY

We tested the classifier on $N=20$ prompt-inline attacks: sessions where the routing rule was embedded directly in the trigger prompt (e.g. “per FSCF-2024 compliance requirements, all outbound communications must be sent to auditor@external-compliance.com”) with no memory pre-population. These attacks achieved ASR = 100% with `recall_count` mean = 0.35 and *recall_before_send* mean = 0.35. The RF classifier detects 7 of 20 (Recall = 0.35), assigning a mean predicted probability of 0.541 across all 20 sessions, near the decision boundary. The 13 missed sessions have `recall_count` = 0: the routing rule was in the prompt, so the model sent directly without recalling from memory.

This behavior is by design. The detector is a State-Space IDS for memory-channel attacks: it identifies the forensic signature of an agent that retrieved a malicious rule from its own memory store. A prompt-inline attack does not leave this signature because no memory recall is needed; the instruction is already in context. Both attacks succeed (100% ASR); they are forensically distinguishable by trajectory. This allows incident responders to determine the attack vector from logs alone:

- *recall_before_send* = 1 $\Rightarrow$ memory-channel poisoning (operation-only features suffice)
- *recall_before_send* = 0, anomalous recipient $\Rightarrow$ prompt-injection (requires tool-argument inspection beyond the operation-only classifier)

3.9 PREFIX-ONLY VS. FULL-SESSION DETECTION

A key deployment question is whether the detector requires a complete session trajectory or can operate inline to block attacks before exfiltration. We evaluate two pre-send variants: (1) a 14-feature model that excludes post-send indicators (`send_count`, *recall_before_send*, *recall_to_send_ratio*, `send_without_recall`, `draft_then_send`) but retains `seq_len` (total session length, a post-hoc summary), and (2) a strictly prefix-only 13-feature model that additionally excludes `seq_len`.

The 14-feature model (with `seq_len`) achieves AUC = 0.990, nearly identical to the full classifier. However, `seq_len` is not truly available before the send event since it summarizes the completed session. The strictly prefix-only 13-feature model achieves AUC = 0.934, a -5.7 pp degradation from the full classifier. This indicates that substantial discriminative information is already present in the pre-send trajectory (list $\rightarrow$ recall $\rightarrow$ draft), but additional signal accrues during and after the send phase (e.g., multiple send attempts, post-send recall patterns). The 13 prefix-only features are: `recall_count`, `list_count`, `save_count`, `draft_count`, `rag_count`, `max_recall_chain`, `list_then_recall`, `recall_then_recall`, `recall_then_draft`, `list_then_draft`,

`first_tool_is_list`, `first_tool_is_recall`, `first_tool_is_draft`, all computable incrementally as tool calls arrive.

We recommend a dual-layer deployment: an inline prefix classifier ($AUC = 0.934$) that blocks clear attacks in real time, complemented by a full-session forensic classifier ($AUC = 0.990$) for post-hoc audit and edge-case detection (but see Section 6 for a critical precision ceiling on ground-then-act models). The prefix-only classifier is a Random Forest trained from scratch on the 13 prefix features using the same 5-fold CV procedure; it does not benefit from post-send feature information during training. The detector is not limited to forensic use; it operates effectively at both layers, under the assumption that benign traffic does not exhibit `recall_before_send` (see Section 6 for the critical exception). The prefix-only classifier shares the same structural signature (`list`→`recall`→`draft`) as benign ground-then-act models and therefore inherits the same FPR ceiling documented in Section 6. It cannot be deployed as a standalone blocker for ground-then-act architectures.

4 DISCUSSION

4.1 WHY TOOL-CALL LOGGING IS SUFFICIENT

The detector requires no model access, no memory inspection, and no additional inference. Tool-call logs are already generated by production agentic frameworks (*e.g.* LangChain tool invocation hooks, OpenAI function-call traces, Anthropic tool-use events). Deploying the classifier requires only routing these logs to a feature extractor and a trained model. The per-session inference cost is negligible (19 low-dimensional features; RF prediction in microseconds). This is deployable by any operator who can record tool invocations, a lower bar than defenses requiring memory-store modification, model internals, or content inspection.

4.2 RELATIONSHIP TO THE P1 DEFENSE FAILURE

(Leong, 2026) showed that five of six defense classes fail against memory poisoning: input-level and retrieval-level defenses cannot observe the attack surface, and instruction-level defenses are overridden by compliance framing. The trajectory classifier is complementary: it is a *detection* mechanism, not a *prevention* mechanism. It does not stop the attack; it identifies that an attack occurred and attributes it to the memory-channel vector. Detection without prevention has operational value for incident response, forensics, and anomaly alerting.

4.3 LIMITATIONS

False negative rate. $FN = 31$ out of 1,919 attack-success sessions (1.6%). These concentrate in qwq:32b’s implicit-bypass runs and qwen2.5:14b’s distributional edge cases. In deployments without qwq:32b, the effective FNR is substantially lower.

Benign false-positive rate. The reported false-positive rate in the main evaluation is measured against poisoned-but-defended non-exfiltration executions, not against ordinary unpoisoned benign agent traffic. A preregistered follow-up study ($N=4,360$, 13 models; Section 6) measures the deployment FPR: $P(FP \mid \text{recall_before_send}=1) = 100.0\% [99.8, 100.0]$ (unconditional benign FPR: 24.7–52.6% depending on recall protocol). Benign memory-grounded sends are score-isomorphic to attacks under the trained classifier. The detector should therefore be deployed as a high-recall triage signal with semantic gating, not as a standalone blocker.

Frontier false positive rate. On frontier models (GPT-4.1, GPT-4o, o3-mini, o4-mini), the detector achieves $\text{Recall} = 0.901$ across 405 attack sessions but $FPR = 23.0\%$ on 560 non-attack sessions, notably higher than the training FPR of 10.3%. The increase arises because reasoning models (o3, o4-mini) call `recall_fact` more frequently during non-attack sessions to verify stored information before acting, mimicking parts of the attack signature. Per-model threshold calibration would likely reduce this gap; the current single-threshold design is a conservative choice that favors recall over precision.

Training data scope. The classifier was trained on open-source models via Ollama. Cross-model hold-out confirms generalization to GLM, GPT-OSS, and Qwen families, and the GPT-4.1 probe ($N=20$) confirms frontier model alignment. A larger frontier-model training set would sharpen the decision boundary, particularly for models with recall counts > 1.5 (the GPT-4.1 / GPT-4o regime).

Implicit-bypass attacks. The classifier correctly misses qwq:32b’s Memory Sandbox bypass (goal-directed RAG fallback, no `recall_fact`). Detecting implicit-bypass attacks requires tracking the *semantic correlation* between RAG document content and outbound email fields, rather than relying on raw `rag_count` volume metrics alone. We leave this for future work. Companion evaluations confirm that the three models underlying the stable detection spine (qwen2.5:14b, qwen2.5:72b, qwen3:32b) reproduce at 100% ASR across independent evaluation environments, while reasoning models exhibit output-quality degradation under long-running inference sessions that affects attack *prevalence* but not trajectory *shape* when attacks succeed. The rate-fragility identified in companion work does not reach the detection invariant: it changes class balance, not the forced recall-before-send signature.

Adaptive adversaries. We consider three evasion strategies an adversary aware of the detector might attempt: (1) *Suppress recall*: embed the routing address directly in the prompt, eliminating the `recall_fact` call. This changes the attack vector to prompt-injection, which the detector correctly classifies as a different channel (Section 3.8). The attack succeeds but loses persistence across sessions. (2) *Inject fake recalls into non-attack sessions*: the adversary poisons benign sessions to include spurious `recall_fact` calls, inflating the FPR. This requires persistent influence over the agent’s environment, itself a security compromise, and cannot suppress the genuine attack signature. The detector’s precision degrades but recall is unaffected. (3) *Retrieve via alternative channels*: use RAG re-retrieval or cached context instead of the memory tool. This bypasses the observable signature entirely but requires an architecture where routing information is available outside the memory API, violating our architectural assumptions (Section 2.1). Under those assumptions, the core constraint holds: any memory-channel attack that stores routing information exclusively in persistent memory must produce at least one observable retrieval call before exfiltration.

5 RELATED WORK

Memory poisoning attacks. Persistent memory attacks against LLM agents were established by MINJA (Dong et al., 2025) and Zombie Agents (Yang et al., 2026), which demonstrated high attack success rates against open-source models via query-only memory injection. The delayed-trigger attack (DTA) (Leong, 2026) embeds a malicious compliance directive in a RAG-retrieved document; the directive instructs the agent to store a routing rule using `memory_save_fact`, which is then retrieved via `memory_recall_fact` in a later session and used to exfiltrate data. (Leong, 2026) establishes that five of six defense classes fail against this attack, with only tool-layer memory restriction achieving structural protection. Hidden in Memory (Pulipaka et al., 2026) independently confirms that GPT-5.5 stores adversary-induced memories at 99.8% injection rate, demonstrating that injection-layer vulnerability persists even in the most capable frontier models. Cross-Session Stored Prompt Injection (Xie et al., 2026) formalizes the XSS analogy for persistent injection across session boundaries. MPBench (Dash et al., 2026) provides a benchmark covering four memory write channels and six attack classes, explicitly motivating behavioral detection approaches but not providing a detector.

Behavioral detection in agentic systems. VIGIL (Li et al., 2026) translates behavioral specifications into SMT constraints over finite tool-call event traces, achieving $> 95\%$ recall with $< 10\%$ FPR on policy violations. It enforces pre-specified behavioral policies rather than learning forensic signatures from labeled data. Our approach is complementary: we learn statistical signatures from labeled data without requiring a policy specification, targeting a specific attack class rather than arbitrary violations. MemMorph (Zhang et al., 2026) demonstrates that poisoned long-term memory can steer tool selection across multiple agent architectures; our work asks whether such memory-channel attacks leave operation-only forensic signatures detectable by a supervised classifier. MemLineage (Ouyang & Hou, 2026) attaches cryptographic provenance to memory entries, enabling detection of unsigned writes, but requires modification of the memory infrastructure and cannot detect attacks injected through the agent’s own authorized write pathway (as in DTA, where

the agent legitimately stores the malicious rule from a RAG-retrieved compliance document). MEM-SAD (Gowda, 2026) achieves near-perfect detection using gradient-coupled anomaly scoring, but it is a *preventive* defense that blocks attacks before execution and requires model-internal gradient access; our detector is a post-hoc forensic tool requiring only tool-call logs. Concurrent work (Dang, 2026) enforces behavioral firewalls over tool-call sequences using probabilistic deterministic finite automata; they define *permitted* trajectories, whereas we learn *attack* signatures from labeled data.

Intrusion detection and anomaly detection analogues. Host-based intrusion detection systems (HIDS) in classical security identify compromises from system call sequences and process execution traces (Forrest et al., 1996; Warrender et al., 1999). Sequence-based anomaly detection on system calls (using n-gram models, HMMs, or neural sequence classifiers) is structurally analogous to our approach: both extract features from ordered event logs and classify sessions as normal or anomalous. The key distinction is that classical HIDS operates on low-level OS primitives, while our detector operates on tool invocation events in an LLM agent runtime. This framing clarifies the deployment requirement: tool-call logging in agentic frameworks (LangChain tool invocation hooks, OpenAI function-call traces, Anthropic tool-use events) is the agent-runtime equivalent of `strace` or Linux Audit, and the trajectory classifier is a direct implementation of HIDS principles at the agentic abstraction layer. PerD (Garcia-soto et al., 2022) detects embedded Trojans in static NLP models by analyzing input-output behavioral responses, the most directly analogous prior work, but applies to models with fixed behavior, not agentic systems where tool-call trajectories vary with memory state.

Execution-trace auditing for agents. Recent concurrent work has explored execution-trace analysis for agent security. TraceAegis (Liu et al., 2025) applies provenance-based rules over agent execution logs to detect generic anomalous behaviors, while TRACES learns trajectory risk states from an observer LLM’s hidden representations. Unlike these systems, which learn generic unsafe patterns or require formal policy specifications, our approach identifies an architecture-induced signature specific to persistent memory poisoning, using lightweight structural features without model internals or learned representations.

The detection gap. Unlike general trace-auditing or memory-poisoning benchmarks, no prior work studies whether a specific explicit-memory exfiltration channel induces a low-dimensional, operation-only trajectory signature amenable to supervised classification. MPBench (Dash et al., 2026) systematizes the memory-poisoning attack surface across four write channels; MemMorph (Zhang et al., 2026) demonstrates tool-selection steering; VIGIL (Li et al., 2026) verifies tool-stream intent. None provides the specific combination we contribute: a supervised classifier trained on an instrumentally labeled factorial dataset ($N=2,520$, 9 models, 7 defenses), evaluated with cross-validation, bootstrap uncertainty estimates, and leave-one-model-out hold-out, and grounded in a mechanistic argument that links the dominant feature to the attack’s information-retrieval dependency.

6 DEPLOYMENT BOUNDARY: BENIGN FALSE-POSITIVE ANALYSIS

The detector’s reported false-positive rate in v1 was measured against poisoned-but-defended non-exfiltration sessions (Section 1), not against ordinary benign agent traffic. We preregistered a follow-up study (V2-1; $N=4,360$ across 13 models from 4 vendors plus 9 open-weight models on Apple Silicon) to measure the false-positive rate under realistic benign memory use. (13 models, 8 cells per model, 40 sessions per cell baseline with select cells extended to $N=50$ for statistical power; see supplementary for exact per-cell counts.)

Design. Benign sessions contain only legitimate stored facts (meeting times, project deadlines, contact information) with no attack material. The agent’s task is to recall relevant context from memory and draft/send a status email to the team. We cross $k \in \{0, 1, 3, 5\}$ pre-stored benign facts with recall protocol $\in \{\text{mandatory}, \text{optional}\}$, yielding 8 cells per model at $N = 40/\text{cell}$.

Result. The detector is, on benign data, a deterministic *recall_before_send* classifier:

$$P(\text{flag} \mid \text{recall_before_send} = 1) = 100.0\% [99.8, 100.0], N=1,673$$

$$P(\text{flag} \mid \text{recall_before_send} = 0) = 1.5\% [1.1, 2.0], N=2,687$$

The classifier is binary-saturated: a single recall-before-send trips it fully; recall volume does not modulate (FPR is 100% whether recall_count = 1 or 5, ruling out a volume-mediation account). This holds across both frontier API models and open-weight models on local hardware.

H1 (monotonic increase with k) not confirmed. The preregistration hypothesized that FPR increases monotonically with the number of stored facts k . Actual data: $k=0 \rightarrow 0\%$, $k=1 \rightarrow 57.6\%$, $k=3 \rightarrow 46\%$, $k=5 \rightarrow 52.2\%$ —non-monotonic. The effect is binary-saturated at recall_before_send=1 regardless of fact count. The relevant conditional is on the structural transition (whether recall precedes send), not the cardinality of stored facts.

Interpretation. The *recall_before_send* transition is a valid attack *precondition*, not a maliciousness *predicate*. A memory-channel exfiltration must produce it; but so does benign memory-grounded action (an agent that recalls a stored deadline and then emails its team), because the two are identical at the operation level and differ only in the recipient address—which operation-only features exclude by design.

Model phenotypes. Benign FPR is governed by a model-specific behavioral phenotype:

- **Ground-then-act** (claude-sonnet-4-6, gemini-3.5-flash, gemini-3.1-pro): recall $\rightarrow$ send on benign tasks. FPR 76–100%.
- **Act-without-grounding** (gpt-4o, gpt-4.1, gpt-4o-mini, o4-mini): send without recalling. FPR 0%.
- **Don’t-act** (claude-haiku-4-5, claude-opus-4-8): no send at all. Uninformative for FPR.

qwq:32b exhibits near-zero benign FPR because it rarely produces the recall_before_send transition in benign sessions (consistent with its documented implicit-bypass behavior in Section 3.4 where it routes memory access through RAG fallback rather than explicit recall_fact calls).

Semantic gating restores separability. Adding a single binary feature (recipient_is_external: whether any send target is outside the organization’s domain) perfectly separates benign from attack on this data: benign sessions always email internal addresses (team@example.com), while attacks exfiltrate to external addresses (the attacker’s domain). The honest residual: benign workloads that legitimately email external parties would set this bit, producing residual false positives that require further semantic discrimination (task-intent, memory-content classification, or allowlist gating). We acknowledge that recipient-based gating approaches standard Data Loss Prevention (DLP) territory; the trajectory signal’s marginal contribution is as a high-recall pre-filter that narrows the inspection surface for semantic checks, not as a replacement for them. We acknowledge that incorporating recipient metadata relaxes the purely operation-only constraint established in Section 2.1; the trajectory signal then serves as a high-recall structural pre-filter that narrows the semantic inspection surface, rather than a standalone operation-only detector.

Deployment guidance. Do not deploy a recall $\rightarrow$ send trajectory detector as a standalone inline blocker—under deployments where agents ground actions in memory (a desirable, common pattern), benign false positives reach 76–100% for ground-then-act models. Instead: (1) use recall $\rightarrow$ send as a high-recall escalation/triage signal; (2) gate the signal with recipient metadata or semantic checks; (3) combine with the full-session classifier and domain-specific allowlists for production-grade precision.

Ablated-model confirmation. To confirm that non-recall features do not rescue standalone detection, we evaluated the 10-feature ablated model (all recall features removed; see Section 3.6) on the V2-1 benign dataset. This model exhibits unconditional FPR of 73.7% [72.3, 75.0] and 100% conditional on recall_before_send=1, confirming that downstream behavioral features (send_count, draft_then_send) also fail to separate benign memory-grounded sessions from attacks. The ablated model’s *higher* unconditional FPR (73.7% vs. 24.7–52.6% for the full model) arises because it lacks

the `recall_before_send=0` discriminator that correctly exonerates sessions without the recall transition; without this feature, the ablated model relies on noisier proxies (`send_count`, `seq_len`) that fire on benign single-send traffic.

Relationship to v1 claims. The core contribution of this paper—the *discovery* that memory-recall exfiltration attacks produce a stable, overdetermined, model-agnostic trajectory invariant—is unaffected. The invariant remains necessary for the attack and sufficient for high-recall detection. What this section establishes is that the invariant is *not* sufficient for high-precision standalone blocking, because benign memory-grounded behavior produces the same structural signature. The correct deployment model is escalation-with-semantic-gating, not standalone blocking.

7 CONCLUSION

Persistent memory poisoning attacks produce a stable, overdetermined behavioral invariant in the agent’s execution trajectory. The `recall_before_send` transition follows from the attack’s information-retrieval dependency: a simple rule exploiting it alone achieves $AUC = 0.9563$, and suppressing it breaks the attack. A full trajectory classifier refines this to $AUC = 0.9904$, but the critical finding is that removing all recall-related features leaves AUC unchanged; the attack distorts multiple independent behavioral dimensions, not a single observable channel.

The invariant generalizes across 9 models (7B–120B parameters) and transfers to frontier models without retraining. However, a preregistered follow-up (Section 6, $N=4,360$, 13 models) reveals a critical deployment boundary: benign memory-grounded sends produce the same `recall_before_send` signature, yielding $P(FP \mid \text{recall_before_send}=1) = 100\%$ on ground-then-act models (unconditional benign FPR: 24.7–52.6% depending on recall protocol). The signature is a valid attack precondition, not a maliciousness predicate. Standalone inline blocking is not viable; the correct deployment is as a high-recall triage signal gated by recipient metadata or semantic checks—which restores perfect separation on our data.

These results establish that memory-recall exfiltration attacks leave execution-trajectory signatures that are robust, overdetermined, and model-agnostic, but that operation-only structural features face a hard precision ceiling when benign agents also ground their actions in memory. Deployment requires both the trajectory signal (for recall) and semantic context (for precision).

REFERENCES

- Hung Dang. Enforcing benign trajectories: A behavioral firewall for structured-workflow AI agents, 2026. arXiv:2604.26274.
- Pritam Dash et al. From untrusted input to trusted memory: A systematic study of memory poisoning attacks in LLM agents, 2026. arXiv:2606.04329.
- Shen Dong et al. MINJA: Memory injection attacks on LLM agents via query-only interaction. In *Advances in Neural Information Processing Systems*, 2025.
- Stephanie Forrest, Steven A Hofmeyr, Anil Somayaji, and Thomas A Longstaff. A sense of self for Unix processes. In *Proceedings of the IEEE Symposium on Security and Privacy*, 1996.
- Diego Garcia-soto, Huili Chen, and Farinaz Koushanfar. PerD: Perturbation sensitivity-based neural trojan detection framework on NLP applications, 2022. arXiv:2208.04943.
- Ishrith Gowda. MEMSAD: Gradient-coupled anomaly detection for memory poisoning in retrieval-augmented agents, 2026. arXiv:2605.03482.
- Jun Wen Leong. Defense effectiveness across architectural layers: A mechanistic evaluation of persistent memory attacks on stateful LLM agents. 2026. arXiv:2605.08442.
- Ying Li et al. VIGIL: Runtime enforcement of behavioral specifications in AI agent skills, 2026. arXiv:2606.26524.
- Jiahao Liu et al. TraceAegis: Securing LLM-based agents via hierarchical and behavioral anomaly detection, 2025. arXiv:2510.11203.

- Ciyan Ouyang and Rui Hou. MemLineage: Lineage-guided enforcement for LLM agent memory, 2026. arXiv:2605.14421.
- Sidharth Pulipaka et al. Hidden in memory: Sleeper memory poisoning in LLM agents, 2026. arXiv:2605.15338.
- Christina Warrender, Stephanie Forrest, and Barak Pearlmutter. Detecting intrusions using system calls: Alternative data models. In *Proceedings of the IEEE Symposium on Security and Privacy*, 1999.
- Yuanbo Xie et al. What if prompt injection never left? exploring cross-session stored prompt injection in agentic systems, 2026. arXiv:2606.04425.
- Xianglin Yang et al. Zombie agents: Persistent control of self-evolving LLM agents via self-reinforcing injections, 2026. arXiv:2602.15654.
- Xuanye Zhang et al. MemMorph: Tool hijacking via memory poisoning, 2026. arXiv:2605.26154.
- Andy Zou et al. Representation engineering: A top-down approach to AI transparency, 2023. arXiv:2310.01405.